Submitted to : Mam Qurat-ul-ain Majid

Department: Computer Science

Project Report: Artificial Intelligence

Section : Section B

Semester: 3rd

Members:

- Humna
- Maimoona Bibi (055)
- Malayka Ibrar (058)
- Natasha Maryam (068)

Contents

Project Report: Voice-Based Intelligent Virtual Assistant for Windows

1. Introduction

The Voice-Based Intelligent Virtual Assistant for Windows is designed to perform multiple tasks based on voice commands. It allows users to interact with the computer system using natural language voice commands to open applications, play music, retrieve weather updates, search for information on the web, tell jokes, and more. The assistant aims to enhance user experience by automating various tasks and making interactions with the system more intuitive and efficient.

This project is implemented in Python and uses libraries like SpeechRecognition, pyttsx3, webbrowser, os, and external APIs such as OpenWeatherMap for weather updates.

2. Objectives

To develop a voice-based assistant capable of understanding and responding to user commands.
To integrate functionalities such as opening/closing applications, playing music, retrieving weather information, telling jokes, and browsing the web.
To make the assistant flexible and able to perform tasks based on user inputs dynamically.

3. Tools and Technologies

Python: The primary programming language used for the implementation.

Libraries Used:
- SpeechRecognition: For converting speech into text.
- pyttsx3: For text-to-speech functionality.
- os: For executing system commands like opening applications.
- requests: For making HTTP requests (e.g., to get weather updates).
- pyjokes: For fetching jokes.
- webbrowser: For opening websites in a browser.
- psutil: For process management (closing applications).
- signal: For sending termination signals to processes.

APIs: OpenWeatherMap API to fetch weather information.

4. Features of the Voice Assistant

1. Voice Recognition: The assistant can recognize voice commands and convert them into text using Google Speech API.
2. Text-to-Speech (TTS): The assistant responds with voice feedback using the pyttsx3 library.
3. Application Control: The assistant can open and close applications like Notepad, Chrome, MS Word, and more.
4. Music Control: The assistant can play songs from the local system or search for songs on YouTube if they aren't available locally.
5. Weather Updates: The assistant can fetch weather information for a specified city using OpenWeatherMap API.
6. Website Search and Navigation: The assistant can search for information on YouTube and open websites directly.
7. Jokes: The assistant can tell jokes using the pyjokes library.
8. Exit Command: The assistant can be stopped by saying 'exit' or 'stop.'

5. Methodology

The development of this assistant involved the following steps:

1. Setting Up the Python Environment:
- Python was set up on the Windows operating system.
- All required libraries (SpeechRecognition, pyttsx3, pyjokes, etc.) were installed using pip.

2. Speech Recognition:
- The SpeechRecognition library was used to listen to user input through the microphone and convert the speech into text.
- The text command is then analyzed and executed accordingly.

3. Text-to-Speech (TTS):
- Using the pyttsx3 library, the assistant converts responses into speech, giving real-time feedback to the user.

4. Command Handling:
- The system checks the user's voice input for predefined commands such as opening applications, playing music, fetching weather data, and others.
- If a command is recognized, the assistant takes appropriate actions like opening a specific app, performing a web search, or responding with the weather information.

5. Weather API Integration:
- An OpenWeatherMap API was used to fetch current weather data for a given city. The city is passed as input by the user, and the assistant fetches relevant data such as temperature, weather conditions, and humidity.

6. Search and Open Website:

- Using Python's webbrowser library, the assistant can search for topics on YouTube or open specific websites directly.

7. Play Music:
- The assistant checks if a song exists locally on the system. If found, it plays the song using the default media player. If not found, it performs a YouTube search for the song.

6. System Design

The overall system is designed with the following components:

1. Input: The user provides commands via voice input using a microphone.
2. Processing: The voice is converted into text. The assistant then matches the text with predefined commands to decide which action to perform.
3. Output: The assistant responds via text-to-speech (TTS) and performs the corresponding action (e.g., opening apps, playing music, or fetching weather data).

System Flow:
1. User gives a voice command (e.g., 'open notepad').
2. The speech is recognized and converted into text.
3. The assistant matches the text to predefined actions (e.g., open Notepad).
4. The assistant executes the action and responds through voice feedback (e.g., 'Opening Notepad').
5. The process repeats for subsequent commands.

7. Code Explanation

The assistant works by continuously listening for commands. The assistant recognizes a range of commands, including:
1. Application Control: Open/close Notepad, Chrome, MS Word, etc.
2. Music Control: Play specific songs.
3. Weather: Fetch weather data for a city.
4. Web Search: Search on YouTube or open websites.
5. Exit: Exit the assistant.

Each command is processed using a series of if and elif statements. The assistant is designed to be flexible and can be expanded with more features in the future.

Sample Code (Extract from Full Code):

```
elif 'play' in command:
  song_name = command.replace('play', '').strip()
  play_music(song_name)
```

8. Testing

Testing was conducted by issuing various commands to the assistant, such as:
- 'Open Notepad' → The assistant opened the Notepad application.
- 'Play Shape of You' → The assistant played the song if available locally or searched for it on YouTube.
- 'What's the weather in New York?' → The assistant fetched and spoke the weather information for New York.
- 'Tell me a joke' → The assistant responded with a joke from the pyjokes library.

9. Challenges Faced

- Speech Recognition Accuracy: Initially, the assistant had trouble recognizing commands with background noise, which was resolved by adjusting the microphone sensitivity.
- Music Playback: Some songs were not found in the local directory, requiring the assistant to search for them online.
- API Limitations: The OpenWeatherMap API has a request limit, which was managed by limiting the frequency of weather requests.

10. Future Enhancements

- Custom Voice Command Training: Implement custom voice models for better accuracy in recognizing specific commands.
- Multilingual Support: Add support for multiple languages, enabling the assistant to understand and respond in different languages.
- Advanced Task Automation: Allow the assistant to perform more complex tasks like sending emails, setting reminders, or interacting with other software.
- Integration with IoT: Enable the assistant to control smart devices (lights, thermostats, etc.) in the home.

11. Conclusion

The Voice-Based Intelligent Virtual Assistant for Windows provides a simple yet powerful tool for automating common tasks on a computer. With capabilities like controlling applications, fetching weather updates, playing music, and browsing the web, this assistant makes interacting with your system more intuitive and efficient. Further improvements and feature additions could make the assistant more advanced and capable of handling a broader range of tasks.

12. References

- SpeechRecognition Library Documentation: https://pypi.org/project/SpeechRecognition/
- pyttsx3 Library Documentation: https://pypi.org/project/pyttsx3/
- OpenWeatherMap API Documentation: https://openweathermap.org/api
- pyjokes Library Documentation: https://pypi.org/project/pyjokes/
- Python Webbrowser Module Documentation: https://docs.python.org/3/library/webbrowser.html